Introducing the Framework for Parsing Plenary Protocols (frappp)

Andreas Blätte (andreas.blaette@uni-due.de)

2018-03-26

A workflow to generate XML

In this vignette, we show how to generate an XML document of a plenary protocol. The example chosen is a verbatim record of a short session of the UN General Assembly. Due to the structural similarity of all kinds of plenary protocols (including the verbatim records of UN bodies), this example may illustrate the workflow for parsing plenary protocols using *frappp*.

The parser of the *frappp* package, i.e. the class *PlenaryProtocolParser* is designed to process plain text documents. However, the usual scenario is that the protocol to be processed is available as a pdf document. In order to walk through the steps we usually encounter, we start with turning a pdf document into plain text.

Preparing a plain text file

There is a set of R packages to extract text from pdf documents, namely Rpoppler and pdftools. However, a typical issue with the pdf of plenary protocols is that documents have a two-column layout. To deal with this special kind of situation, we have developed the package *trickypdf*. It is not yet available at CRAN, but it can be installed in convenient manner from the GitHub presence of the PolMine project using the installation mechanism offered by the *devtools* package.

```
if (!"trickypdf" %in% installed.packages()[,"Package"]){
  if (!"devtools" %in% installed.packages()[,"Package"]) install.packages("devtools")
  devtools::install_github("PolMine/trickypdf")
}
library(trickypdf)
```

```
## Loading required package: knitr
```

```
## Warning: package 'knitr' was built under R version 3.4.3
```

The *trickypdf* package includes some pdf documents serving as example. The document we will process is the first meeting of the 2016 UN General Assembly (A/71/PV.1). In interactive R session, have a look at the document, and consider the two-column layout.

```
doc <- system.file(package = "trickypdf", "extdata", "pdf", "UN_GeneralAssembly_2016.pdf")
if (interactive()) browseURL(doc)
```

With other approaches for processing pdf, you would extract the text from the pdf document and have to invest some time to get rid of anything on the margins of the pdf document as a post-processing step. The *trickypdf* package allows you to define in advance the boxes with relevant information. The package documentation explains in more detail how you would do that.

In our case, the following commands will get us the plain text out of the protocol of the General Assembly.

```
UN <- trickypdf::PDF$new(filename_pdf = doc)
UN$add_box(page = 1, box = c(top = 232, height = 400, left = 52, width = 255))
UN$add_box(page = 1, box = c(top = 232, height = 400, left = 303, width = 255), replace = FALSE)
UN$add_box(page = 2, box = c(top = 80, height = 595, left = 52, width = 255))
```

```
UN$add_box(page = 2, box = c(top = 80, height = 595, left = 303, width = 255), replace = FALSE)
UN$get_text_from_boxes(paragraphs = TRUE)
plaintext <- unname(unlist(UN$pages))
```

The *PlenaryProtocolParser* takes plain text files as input. So we save the document to a temporary file.

```
undoc_tmp <- tempfile()
writeLines(text = plaintext, con = undoc_tmp)
```

From Plain text to XML (TEI standard)

Now we move to processing the document using the framework offered by the package *frappp*.

```
library(frappp)
```

```
## Loading required package: ctk
## Loading required package: data.table
## The registry directory is not defined.
## See ?cqi_setRegistry for more info on how to set the registry.
## ctk 0.2.24
## ... checking for TreeTagger ... FAIL
```

The class that implements the workflow to process a plenary protocol is the *PlenaryProtocolParser* class. It is a class of the R6 class system.

The notation of using R6 classes may be somewhat uncommon for many R users. Usually you will know the S3 and the S4 class system. To learn more about R6 classes, consult the vignette of the R6 package, and the introduction to the different class systems in Hadley Wickham's Advanced R Online Book. It offers an excellent entry to understand the notion of reference classes (there is also a hard copy of the book).

Compared to R's built-in reference classes (RefClass), R6 is more efficient (see the vignette on R6 and Reference class performance tests). But in fact, the biggest advantage of R6 is that you can include classes inheriting from the *PlenaryProtocolParser* class in other packages than *frapp*. R6 allows inheritance across packages. This is essential about this package: It is intended to serve as a basis for projects that develop corpora of plenary protocols.

Now let us instantiate our parser. We will then configure it to process General Assembly verbatim records.

```
PPP <- PlenaryProtocolParser$new()
```

Processing the document requires two steps. First, we have to configure the parser. This means that we supply a set of regular expressions to detect speakers, agenda items and instructions from the stage. Second, we process the individual document.

Configuring the parser

Defining regular expressions to detect speakers

The field `$speakerRegex` supplies the information to detect speakers and to extract information that will go into XML attributes later on. It is a named list of lists that need to offer a regular expression (length 1 character vector), and a function that returns a named character vector with information that will be used to construct XML attributes for speakers later on.

```
PPP$speakerRegex <- list(

  pres = list(
    regex = "^(The\\s+President):\\s*(.+)$",
    fn = function(who, txt){
      c(
        who = who,
        state = "NA",
        role = "presidency",
        position = "presidency",
        text = txt
      )
    }
  ),

  sg = list(
    regex = "^(The\\sSecretary-General):\\s*(.+)$",
    fn = function(who, txt){
      c(
        who = who,
        state = "NA",
        role = "secretary-general",
        position = "secretary-general",
        text = txt
      )
    }
  )

)
```

It is futile to seek for the regular expression that will always match perfectly. Sometimes, our regular expression will match unwantedly. A vector of speaker mismatches will be used to omit unwanted matches. In our case, we do not yet need that, however.

```
PPP$speakerMismatch <- character()
```

Define regular expressions to detect calls for agenda items

Detecting calls to agenda items is optional. The body of a document can also be created with `$makeBody(agendaItems = FALSE)`. For detecting agenda item calls, define the field *agendaItemRegex* as a named character vector of (alternative) regular expressions that will match a call to an agenda item.

```
PPP$agendaItemRegex <- c(any = "^Item\\s+\\d+\\sof.*agenda.*$")
PPP$agendaItemTypes <- c(any = "debate")
```

Define regular expressions to detect stage instructions

To mark-up stage instructions, such as interjections, supply a named character vector with regular expressions. In the document that is generated, the value of the *type* attribute of the *stage* element surrounding what has been matched will be the name of the regular expression.

```
PPP$stageRegex <- c(
  end_of_session = "^\\s*The\\s+meeting\\s+rose\\s+at\\s+\\d+\\s[ap]\\.m\\.\\s*$"
)
```

Processing a Protocol

We are now ready to parse our first plenary protocol. Once the parser has been initialized, the user intervention that is required is minimal. The whole idea behind *frappp* is to automate parsing plenary protocols, thus making the procedure reproducible.

However, the parser requires the metadata of the document to generate a telling header. But its generic version does not have a method to extract the relevant information from the document. So we write a function that does this, based on the original pdf document.

```
getMetadata <- function(filename){
  pdfdoc <- pdftools::pdf_text(pdf = doc)
  page1 <- strsplit(pdfdoc, "\n")[[1]]

  # get session
  docIdRegex <- "^.*?\\s*(A)\\s*/(\\d+)/\\s*(PV\\.\\d+).*?$"
  session <- do.call(rbind, gsubfn::strapplyc(X = page1, pattern = docIdRegex))[1,2]

  # get date
  weekdays = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
  months = c(
    January = 1, February = 2, March = 3, April = 4, May = 5, June = 6,
    July = 7, August = 8, September = 9, October = 10, November = 11, December = 12
    )
  dateRegex <- sprintf(
    "^\\s*(%s),\\s*(\\d{1,2})\\s+(%s)\\s+(\\d{4}),\\s+\\d{1,2}\\.\\d{2}\\s+(a\\.m\\.|p\\.m\\.)\\s*$",
    paste0(weekdays, collapse = "|"),
    paste0(names(months), collapse = "|")
  )
  dateMatch <- do.call(rbind, gsubfn::strapply(page1, pattern = dateRegex, c))
  year <- dateMatch[1,4]
  month <- as.integer(months[dateMatch[1,3]])
  day <- as.integer(dateMatch[1,2])
  dateOfMeeting <- sprintf("%s-%02d-%02d", year, month, day)

  # get meeting
  meetingRegex <- "\\s+(\\d+)(st|nd|rd|th)\\s+plenary\\s+meeting\\s*$"
  meeting <- do.call(rbind, gsubfn::strapplyc(page1, pattern = meetingRegex))[1,1]

  list(
    legislativePeriod = as.integer(session),
    sessionNo = as.integer(meeting),
    date = as.Date(dateOfMeeting),
    retrievalDate = "unknown",
    filetype = "pdf",
    url = "http://www.un.org/en/ga/",
    publisher = "United Nations",
    title = "Verbatim Records of the United Nations"
  )
}
```

Note that the original pdf document is the basis for getting the metadata: The metadata is found in regions that we wanted to get rid to have the clean text body.

```
metadata <- getMetadata(doc)
```

Let's go. The first step is to read a file. We use the plain text file we used before.

```
PPP$readFile(filename = undoc_tmp, id = "A/71/PV.1")
```

Second. The initialized document includes some default metadata values: The birthday the document was parsed, the package used ("frappp") and the version of the package (see `PPP$metadata`). We add the metadata extracted before to these default metadata values.

```
PPP$addMetadata(data = metadata)
```

Third. We now generate the base TEI document by inserting the metadata into the TEI header. To do this, the parser uses the names of the character vector in the metadata to find matching XPath expressions in a character vector in the field xpath of the parser (`PPP$xpath`), using names. The respective XPath expression is used to find the correct place in the XML document for the piece of information of the metadata.

After running `$makeHeader()`, the field xml will contain the TEI document with the populated header (see `PPP$xml`, or `cat(as.character(PPP$xml))`).

```
PPP$makeHeader()
```

Third. The raw text (in the field `PPP$txt`) may include some noise. A pre-processing method (`$preprocessing()`) will remove empty lines, and leading and trailing whitespace. It takes as input a list `replacements`, a list of length 2 character vectors that serve as input to `gsub` (parameters pattern and x, respectively), to perform usual clean-ups.

```
PPP$preprocessing() # remove leading and trailing spaces
```

Fourth. The essential feature of a plenary protocol is that it reports speech attributed to speakers. Indicating who is speaking is the organising principle of a protocol. So we use the regular expressions in the field `PPP$speakerRegex` to split up the (pre-processed) plain text into chunks of speech that will be kept (as a list) in the field `$chunks`.

```
PPP$splitBySpeakers()
```

If you look at any of these chunks (e.g. `PPP$chunks[[2]]`), you will see that the text has been extracted nicely from the pdf document, but that paragraphs have been destroyed.

Fifth: We reconstruct the paragraphs!

```
PPP$reconstructParagraphs(exclude = "pres")
```

See `PPP$chunks[[2]]` to see the result of this procedure. When reconstructing paragraphs, the heuristic is that lines are concatenated, when their length undercuts a standard length. To guess the number of characters per line that might be optimal, the method `$showNcharHistogram()` can be used.

Sixth. All preparatory steps have been performed. We now insert the speakers and the text of their speeches into the body of the XML document already present in the `$xml` field.

```
PPP$makeBody(agendaItems = FALSE)
```

That's it! The XML document is in the `$xml` field of our class. You can either look at it by typing `cat(as.character(PPP$xml))`, if by using the convenience of a browser that is called with the `$browseXML()` method.

```
if (interactive()) PPP$browseXML()
```

To write the XML document to a file, you can use the following snippet.

```
outfile <- tempfile()
writeLines(text = as.character(PPP$xml), con = outfile)
```

This has been the verbose version of parsing a protocol. In real life, you certainly want things to be more

efficient. So note that the implementation of the PlenaryProtocolParser meets the requirements to use a technique called method chaining, which is very similar to the pipes more commonly known in R.

```
PPP$readFile(filename = undoc_tmp, id = "A/71/PV.1")$
  addMetadata(data = getMetadata(doc))$
  makeHeader()$
  preprocessing()$
  splitBySpeakers()$
  reconstructParagraphs(exclude = "pres")$
  makeBody(agendaItems = FALSE)$
  saveXML(filename = tempfile())
```

Defining classes inheriting from PlenaryProtocolClass

It will depend on your corpus preparation scenario whether it makes sense to configure the PlenaryProtocolParser as it is, or whether it is better to define a class inheriting from it. We want to illustrate how a specified class to process the verbatim records of the UN General Assembly might look like. To be able to define a class inheriting from PlenaryProtocolClass, we need to load the R6 package.

```
library(R6)
```

The VerbatimRecordUNGA class we now define does not contain any code that you have not yet seen before.

(a) It includes an `$initialize()` method that will supersede the respective method of the parent class. The idea is that upon initialization, the regular expressions we know to work (or assume to work) are assigned to the respective fields.

(b) The `$getMetadata()` function we developed previously is now a method of the new class.

(c) The class includes a `$xmlify()` method that takes the files needed as input (filenames of pdf, txt, xml files), and that will perform the succession of steps to prepare the XML document.

It goes beyond this vignette to explain the details of OOP. Hadley Wickham's (online) book provides a good introduction to the R6 class system.

```
VerbatimRecordUNGA <- R6Class(

  "VerbatimRecordUNGA",
  inherit = PlenaryProtocolParser,

  public = list(

    initialize = function(){

      self$xpath = teiXpath

      self$speakerRegex <- list(
        pres = list(
          regex = "^(The\\s+President):\\s*(.+)$",
          fn = function(who, txt){
            c(
              who = who,
              state = "NA",
              role = "presidency",
              position = "presidency",
              text = txt
```

```
        )
      }
    ),
    sg = list(
      regex = "^(The\\sSecretary-General):\\s*(.+)$",
      fn = function(who, txt){
        c(
          who = who,
          state = "NA",
          role = "secretary-general",
          position = "secretary-general",
          text = txt
        )
      }
    )
  )
  self$speakerMismatch <- character()
  self$agendaItemRegex <- c(any = "^Item\\s+\\d+\\sof.*agenda.*$")
  self$agendaItemTypes <- c(any = "debate")
  self$stageRegex <- c(
    end_of_session = "^\\s*The\\s+meeting\\s+rose\\s+at\\s+\\d+\\s[ap]\\.m\\.\\s*$"
  )

  invisible(self)
},


getMetadata = function(filename){
  pdfdoc <- pdftools::pdf_text(pdf = doc)
  page1 <- strsplit(pdfdoc, "\n")[[1]]

  # get session
  docIdRegex <- "^.*?\\s*(A)\\s*/(\\d+)/\\s*(PV\\.\\d+).*?$"
  session <- do.call(rbind, gsubfn::strapplyc(X = page1, pattern = docIdRegex))[1,2]

  # get date
  weekdays = c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
  months = c(
    January = 1, February = 2, March = 3, April = 4, May = 5, June = 6,
    July = 7, August = 8, September = 9, October = 10, November = 11, December = 12
  )
  dateRegex <- sprintf(
    "^\\s*(%s),\\s*(\\d{1,2})\\s+(%s)\\s+(\\d{4}),\\s+\\d{1,2}\\.\\d{2}\\s+(a\\.m\\.|p\\.m\\.)\\s*$
    paste0(weekdays, collapse = "|"),
    paste0(names(months), collapse = "|")
  )
  dateMatch <- do.call(rbind, gsubfn::strapplyc(page1, pattern = dateRegex))
  year <- dateMatch[1,4]
  month <- as.integer(months[dateMatch[1,3]])
  day <- as.integer(dateMatch[1,2])
  dateOfMeeting <- sprintf("%s-%02d-%02d", year, month, day)

  # get meeting
```

```
      meetingRegex <- "\\s+(\\d+)(st|nd|rd|th)\\s+plenary\\s+meeting\\s*$"
      meeting <- do.call(rbind, gsubfn::strapplyc(page1, pattern = meetingRegex))[1,1]

      list(
        legislativePeriod = as.integer(session),
        sessionNo = as.integer(meeting),
        date = as.Date(dateOfMeeting),
        retrievalDate = "unknown",
        filetype = "pdf",
        url = "http://www.un.org/en/ga/",
        publisher = "United Nations",
        title = "Verbatim Records of the United Nations"
      )
    },

    xmlify = function(id, txtfile, pdffile, xmlfile, verbose = TRUE){
      if (verbose) message("... getting metadata")
      metadata <- self$getMetadata(pdffile)
      if (verbose) message("... reading plain text file: ", txtfile)
      self$readFile(filename = txtfile, id = id)
      if (verbose) message("... adding metadata that has been extracted")
      self$addMetadata(data = metadata)
      if (verbose) message("... making header")
      self$makeHeader()
      if (verbose) message("... pre-processing plain text")
      self$preprocessing()
      if (verbose) message("... splitting up by speakers")
      self$splitBySpeakers()
      if (verbose) message("... reconstructing paragraphs")
      self$reconstructParagraphs(exclude = "pres")
      if (verbose) message("... making body")
      self$makeBody(agendaItems = FALSE)
      if (verbose) message("... saving TEI document to file: ", teifile)
      self$saveXML(filename = xmlfile)
      invisible(self$xml)
    }
  )
)
```

Having the defined our specified class for processing UN General Assembly documents, much less code is required to process a single document.

```
UNGAParser <- VerbatimRecordUNGA$new()
teifile <- tempfile()
tei <- UNGAParser$xmlify(id = "A/71/PV.1", txtfile = undoc_tmp, pdffile = doc, xmlfile = teifile)
```

To check whether the result is ok, use `cat(as.character(tei))`.

Processing multiple documents

Usually, we want to process many protocols, not just one. Having prepared a class that is specialized to process a specific type of plenary protocol is the basis for moving to batch processing multiple documents. The following block of code may serve as an example how we might loop over multiple documents.

```
pdfdir <- tempdir()
txtdir <- tempdir()
xmldir <- tempdir()
txtFilesToProcess <- Sys.glob(paths = sprintf("%s/*.txt", txtdir))

UNGAParser <- VerbatimRecordUNGA$new()

lapply(
  txtFilesToProcess,
  function(txtfile){
    UNGAParser$xmlify(
      id = basename(txtfile),
      txtfile = txtfile,
      pdffile = file.path(pdfdir, gsub("^(.*?)\\.txt$", "\\1.pdf", basename(txtfile))),
      xmlfile = file.path(pdfdir, gsub("^(.*?)\\.txt$", "\\1.xml", basename(txtfile)))
    )
    return(TRUE)
  })
```

```
## list()
```

Enjoy using *frappp*!

Annexes

Annex 1: The TEI template

```
teiTemplate <- xml2::read_xml(system.file("xml", "teiTemplate.xml", package = "frappp"))
cat(as.character(teiTemplate))
```

```
## <?xml version="1.0" encoding="UTF-8" standalone="no"?>
## <TEI>
##   <teiHeader>
##     <fileDesc>
##       <titleStmt>
##         <title/>
##         <legislativePeriod/>
##         <sessionNo/>
##       </titleStmt>
##       <editionStmt>
##         <edition>
##           <package/>
##           <version/>
##           <birthday/>
##         </edition>
##       </editionStmt>
##       <publicationStmt>
##         <publisher/>
##         <date when=""/>
##         <page/>
##       </publicationStmt>
##       <sourceDesc>
##         <filetype/>
```

```
##         <url/>
##         <date/>
##       </sourceDesc>
##     </fileDesc>
##     <encodingDesc>
##       <projectDesc>PolMine-Project (www.polmine.de)</projectDesc>
##       <samplingDecl/>
##       <editorialDecl/>
##     </encodingDesc>
##     <profileDesc/>
##     <revisionDesc/>
##   </teiHeader>
##   <text>
##     <body/>
##   </text>
## </TEI>
```

Annex 2: Default XPath expressions

```
teiXpath
```

```
## $title
## [1] "//TEI/teiHeader/fileDesc/titleStmt/title"
##
## $legislativePeriod
## [1] "//TEI/teiHeader/fileDesc/titleStmt/legislativePeriod"
##
## $sessionNo
## [1] "//TEI/teiHeader/fileDesc/titleStmt/sessionNo"
##
## $date
## [1] "//TEI/teiHeader/fileDesc/publicationStmt/date"
##
## $publisher
## [1] "//TEI/teiHeader/fileDesc/publicationStmt/publisher"
##
## $url
## [1] "//TEI/teiHeader/fileDesc/sourceDesc/url"
##
## $filetype
## [1] "//TEI/teiHeader/fileDesc/sourceDesc/filetype"
##
## $retrievalDate
## [1] "//TEI/teiHeader/fileDesc/sourceDesc/date"
##
## $package
## [1] "//TEI/teiHeader/fileDesc/editionStmt/edition/package"
##
## $version
## [1] "//TEI/teiHeader/fileDesc/editionStmt/edition/version"
##
## $birthday
## [1] "//TEI/teiHeader/fileDesc/editionStmt/edition/birthday"
```

Annex 2: The final short TEI document

```
cat(as.character(tei))
```

```
## <?xml version="1.0" encoding="UTF-8" standalone="no"?>
## <TEI>
##   <teiHeader>
##     <fileDesc>
##       <titleStmt>
##         <title>Verbatim Records of the United Nations</title>
##         <legislativePeriod>71</legislativePeriod>
##         <sessionNo>1</sessionNo>
##       </titleStmt>
##       <editionStmt>
##         <edition>
##           <package>frappp</package>
##           <version>0.4.0</version>
##           <birthday>2018-03-26</birthday>
##         </edition>
##       </editionStmt>
##       <publicationStmt>
##         <publisher>United Nations</publisher>
##         <date when="">2016-09-13</date>
##         <page/>
##       </publicationStmt>
##       <sourceDesc>
##         <filetype>pdf</filetype>
##         <url>http://www.un.org/en/ga/</url>
##         <date>unknown</date>
##       </sourceDesc>
##     </fileDesc>
##     <encodingDesc>
##       <projectDesc>PolMine-Project (www.polmine.de)</projectDesc>
##       <samplingDecl/>
##       <editorialDecl/>
##     </encodingDesc>
##     <profileDesc/>
##     <revisionDesc/>
##   </teiHeader>
##   <text>
##     <body>
##       <sp who="The President" state="NA" role="presidency" position="presidency">
##         <speaker>The President:</speaker>
##         <p>I now give the floor to the Secretary- General, His Excellency Mr. Ban Ki-moon. The Secre
##       </sp>
##       <sp who="The President" state="NA" role="presidency" position="presidency">
##         <speaker>The President:</speaker>
##         <p>I thank the Secretary-General for  his statement. Item 138 of the provisional agenda Scal
##         <p>Item 7 of the provisional agenda Organization of work of the seventy-first session of  th
##       </sp>
##       <sp who="The President" state="NA" role="presidency" position="presidency">
##         <speaker>The President:</speaker>
##         <p>The Assembly will now turn its  attention to document A/71/382, which contains a  letter
##         <stage type="end_of_session">The meeting rose at 4 p.m.</stage>
```

```
##       </sp>
##     </body>
##   </text>
## </TEI>
```